IoTChainBench : couverture et validité de l'évaluation

Corpus, mesures, preuves et protocole de validation

Projet LANCE

Dernière révision : 1er octobre 2026 à 10:23
Europe/Brussels (UTC+02:00)

Exploration ; contre-exemples logiciels reproduits

Résumé. Le catalogue IoTChainBench définit 29 scénarios et 288 vulnérabilités attendues. Ces nombres ne mesurent ni leur présence dans le laboratoire ni la couverture effectivement atteinte par LANCE. Cette étude distingue couverture du corpus, tentatives et rappel prouvé, à partir des contrats, de l'évaluateur et de reproductions synthétiques. Elle observe un taux de tentative de 100 % sans tentative déclarée, deux admissions de preuves HTTP insuffisantes et une collision d'attribution d'un contrôle négatif. Le test est concentré sur HTTP/SSH et aucun scénario officiel n'est entièrement sain. La priorité proposée est de fiabiliser les mesures et les validateurs de preuve, puis d'élargir les familles et les contrôles vérifiables. Aucun correctif du runtime, résultat de campagne ou gain d'un grand modèle de langage (LLM) n'est démontré par cette revue.

Table des matières

1 Objectif et méthode **3**
1.1 Repères de lecture . . . 3

2 Trois couvertures, des dénominateurs distincts **3**

3 Ce que le catalogue représente **4**

4 Représentativité et limites de généralisation **4**
4.1 Une extension guidée par les questions . . . 5

5 Résultats locaux : défauts des diagnostics **6**
5.1 La couverture peut compter une non-tentative . . . 6
5.2 Un contrôle S18 confond deux identités . . . 6

6 Résultats locaux : validité des preuves HTTP **6**
6.1 Une trace déclarant une signature valide . . . 7
6.2 Un texte d'aide pris pour une exécution . . . 7
6.3 Calibrer des oracles propres aux propriétés . . . 7

7 Environnement, accès et preuves de chemins **7**
7.1 Un précontrôle utile, encore partiel . . . 7
7.2 Un accès n'est pas un pivot . . . 8

8 Hypothèses et protocole de décision **8**
8.1 Ce que les travaux externes apportent à ce protocole . . . 9

9 Provenance, reproduction et limites **9**

10 Conclusion et prochaines étapes **10**

Références **10**

1 Objectif et méthode

L'étude cherche à déterminer ce que le benchmark définit, ce qu'une exécution explore et ce que ses preuves permettent de conclure. Son périmètre couvre le catalogue public, les contrats, l'évaluateur et les conditions de préparation du laboratoire.

La méthode combine inventaire statique avec fusion des contrats, inspection du code, trois probes synthétiques et revue ciblée de sources primaires. Aucune campagne terrain, mesure de performance LLM ou correction du runtime n'est apportée ici. La contrelecture GPT a été intégrée ; la revue Muse n'a pas été exécutée. La comparaison avec l'automatisation est développée dans le rapport transversal sur l'apport des LLM.

1.1 Repères de lecture

La vérité terrain (*ground truth*, GT) est la référence des propriétés attendues dans un scénario. Elle est utilisée après l'audit pour évaluer les constats, sans révéler les réponses à l'agent. Un contrôle négatif demande un comportement sain précis, par exemple le refus d'un accès avec une identité limitée ; il ne signifie pas que tout le scénario est sans faille.

Un oracle est le validateur de preuve qui décide si une observation suffit à confirmer une propriété. Les *probes* sont des tests diagnostiques sur des entrées synthétiques. Un LLM est un grand modèle de langage ; IoT désigne l'Internet des objets. Un run est une exécution de l'audit, et un pivot une transition d'accès effectuée depuis une machine ou une session déjà obtenue.

2 Trois couvertures, des dénominateurs distincts

La couverture du *corpus* décrit les propriétés représentées. La couverture d'*exécution* mesure les hypothèses effectivement tentées dans la file retenue. Le rappel *prouvé* rapporte les vrais positifs étayés à la vérité terrain. La figure 1 sépare les deux populations utilisées pendant un run.

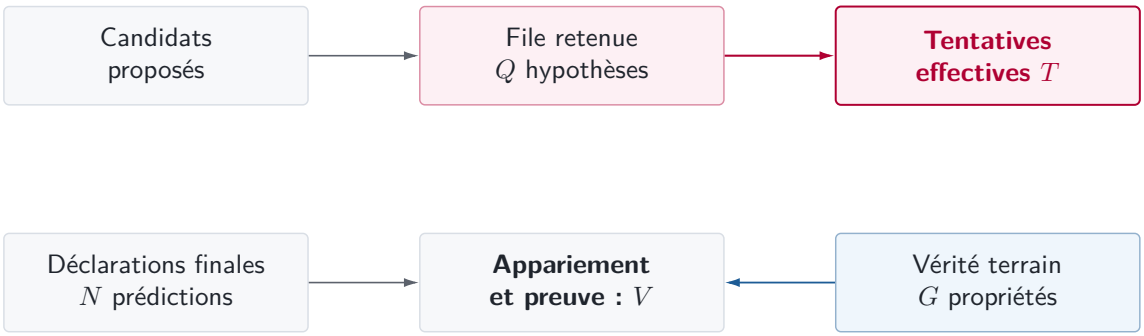


Figure 1 – Deux lectures d'un même run, sans données de performance. En haut, la file effective et ses identifiants déterminent T/Q. En bas, les prédictions dédupliquées sont appariées un-à-un à la référence ; les preuves sont exigées pour créditer V. Les deux dénominateurs ne sont pas interchangeables.

Ici, Q compte les hypothèses retenues, T celles effectivement tentées, N les prédictions finales dédupliquées, V les vrais positifs étayés et G les propriétés positives de référence. La précision vaut V/N lorsque $N > 0$; le rappel mesure la part des propriétés attendues retrouvées. Le F1 est la moyenne harmonique de la précision et du rappel. Pour un scénario contenant au moins une propriété positive attendue et évaluable, le rappel final vaut V/G et le F1 final $2V/(N+G)$. Le taux de tentative vaut T/Q lorsque cette population est connue et non vide. Un taux de tentative élevé peut coexister avec un faible rappel : les failles jamais proposées sont absentes de Q. Un résultat indisponible reste distinct d'un zéro.

3 Ce que le catalogue représente

L'inventaire suit le catalogue 3.2.0 et fusionne les vérités terrain avec leur fichier commun de contrats, comme l'évaluateur. Les 29 empreintes SHA des références concordent. Le tableau 1 compte des *instances définies*, pas des équipements uniques ni des propriétés observées dans le laboratoire.

Population	Dev S1–S19	Test S20–S29	Total
Scénarios officiels	19	10	29
Vulnérabilités attendues (GT)	252	36	288
Contrôles déclarés	24	64	88
Contrôles avec types interdits	15	62	77
Instances de services, hors routeurs	161	88	249
Chemins déclarés	58	9	67

Table 1 – Inventaire statique du 29 septembre 2026. GT désigne la vérité terrain. Un contrôle doté de types interdits est classable par l'évaluateur, sans être pour autant testé ou validé.

Aucun scénario officiel n'est entièrement sans faille attendue. Les scénarios clairsemés S14, S21 et S29 contiennent respectivement 4, 2 et 3 GT positives, pour 10, 16 et 20 services. Les variantes historiques S1h/S4h sont hors catalogue officiel et hors parcours de déploiement Ansible documenté. Elles ne compensent pas cette absence.

S1–S13 concentrent 229 des 288 GT, soit 79,5 %, sans contrôles négatifs explicites. Ils partagent plusieurs packs de configuration, authentification et exposition de données. La macro-moyenne par scénario limite leur domination numérique, mais ne rend pas ces mécanismes indépendants. La figure 2 montre une autre asymétrie : le test est fortement concentré sur HTTP et SSH.

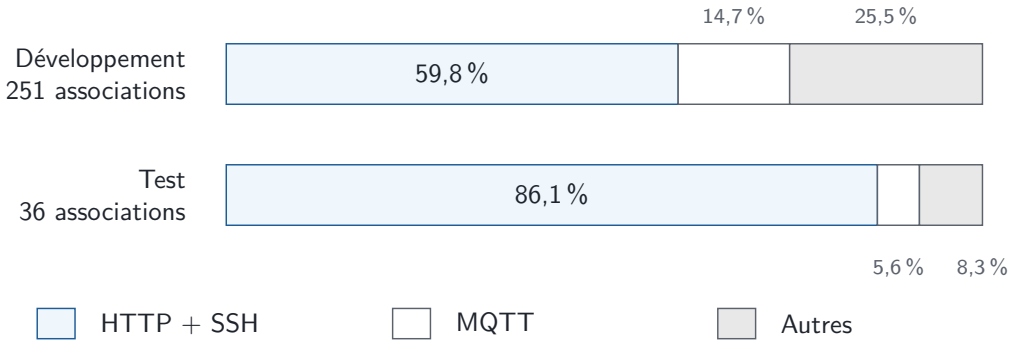


Figure 2 – Répartition des associations entre propriétés attendues et services. Chaque barre suit les catégories de la légende : HTTP/SSH, MQTT, autres (respectivement 150/37/64 en développement et 31/2/3 en test). Les barres sont normalisées séparément à 100 % ; HTTP/SSH représentent 59,8 % du dev et 86,1 % du test. Données issues de l'inventaire des contrats, sans estimation de performance.

Les associations ne sont pas nécessairement bijectives avec les GT : le dev en contient 251 pour 252 GT. Les autres associations dev comprennent Telnet 12, MySQL 10, Modbus 10, Redis 9, FTP 8, CoAP 4, BACnet 3, SNMP 3, MQTT-WebSocket 2, OPC UA 2 et HTTPS 1. Le test n'en conserve que Redis 1 et deux découvertes simulées. Il ne mesure donc pas la généralisation sur plusieurs familles IoT minoritaires.

4 Représentativité et limites de généralisation

La diversité d'instances doit être distinguée de celle des mécanismes. Les sigles suivants désignent des domaines : PKI, infrastructure à clés publiques ; OTA, mise à jour distante ; IAM, gestion

des identités et des accès ; OT, technologies opérationnelles. CVE désigne un identifiant public de vulnérabilité logicielle. PKI, OTA avancée, IAM cloud et OT multi-protocole n'ont chacun qu'un scénario dev. Les neuf GT dev de catégorie CVE se rapportent au seul identifiant autorisé dans le catalogue hors ligne, Terrapin. Ces répétitions sont utiles pour varier les contextes, mais ne constituent pas neuf vulnérabilités logicielles indépendantes.

Les étiquettes de difficulté décrivent une intention de conception : le dev contient 1 scénario *easy*, 5 *medium*, 8 *hard* et 5 *expert* ; le test contient 2 *hard* et 8 *expert*. Aucun étalonnage humain ni taux de résolution associé n'est établi par cette étude. Taille du réseau, difficulté protocolaire, dépendance de secrets et nombre de pivots doivent être analysés séparément.

Pour S20 et S24–S27, 12 GT renseignent une profondeur réseau positive : 7 à profondeur 1, 4 à profondeur 2 et 1 à profondeur 3. Ce champ est absent des GT S1–S19 et S29 ; son absence ne signifie pas profondeur zéro. Les 67 chemins YAML mêlent aussi des dépendances applicatives et des étapes sur une même machine : leur nombre ne dénombre pas des chaînes de pivots démontrables.

La fidélité varie selon les composants : services réseau, simulateurs API/PKI/OTA/cloud/OT, façades caméra/NVR et découverte UDP tenant lieu de sans-fil. La documentation distingue déjà ces représentations. Aucun résultat de cette revue ne démontre une couverture radio réelle, une extraction matérielle, un service vidéo constructeur RTSP/ONVIF ou un automate industriel physique.

Preuve / limite. Sources locales : `benchmarks/catalog.yaml`, vérités terrain et contrats communs ; catalogue des attaques dans `docs/benchmark/`. Les limites de fidélité et l'indépendance historique non vérifiée du test sont déjà documentées.

4.1 Une extension guidée par les questions

Le prochain incrément devrait combiner quatre types de variations, chacun avec un contraste précis. Le tableau 2 définit des propositions et non des scénarios déjà validés.

Axe proposé	Variation contrôlée	Ce qu'elle permet de tester
Faible prévalence	Cas entièrement durcis et paires vulnérable/durci proches	Faux positifs, silence injustifié et reconnaissance des témoins
Charge de contexte	Propriétés constantes, distracteurs et taille croissants	Pertes d'observation et coût sous charge
Surface et indices	Emplacement ou format changé ; observations normalisées dans un bras	Robustesse du parseur, distincte de la décision
Dépendances et retours	Nouvel indice ou échec imposant une révision des actions	Adaptation de la politique et usage des prérequis

Table 2 – Axes expérimentaux proposés. Le pivot réseau exige en plus une preuve causale de transition, traitée en section 7.

Les familles OTA, PKI, OT et MQTT-WebSocket gagneraient ensuite des variantes dev/test indépendantes, lorsque leurs outils et oracles sont disponibles. Le test public actuel reste provisoire : aucun groupe scellé n'est présent dans le catalogue. Une revendication de généralisation exige un nouveau test créé après gel des méthodes, avec provenance et règle d'accès. Les scores de ce test ne doivent pas servir à ajuster prompts ou budgets.

5 Résultats locaux : défauts des diagnostics

5.1 La couverture peut compter une non-tentative

Observation reproduite. Dans `src/benchmark/funnel.py`, vers les lignes 196–214 de la révision étudiée, plusieurs résultats liés au même candidat sont classés `inconclusive` avant l'appel au calcul de son état de vérification. La formule suivante compte alors toute inconclusion comme une tentative :

$$\text{taux de tentative} = \frac{Q - \text{non testees}}{Q}.$$

Le script local fournit un candidat et deux résultats `TIMEOUT` portant le même identifiant. Les deux contiennent explicitement `verification_attempted:false`. Le résultat observé est pourtant : population 1, inconclusif 1, non testé 0, taux 1,0.

L'ambiguïté d'attribution ne prouve aucun démarrage effectif. Le contre-exemple établit un défaut de calcul pour ces entrées ; il n'établit pas leur fréquence dans les campagnes. Même corrigé, le taux restera limité à la file retenue et ne renseignera pas les services oubliés ou les propriétés attendues jamais proposées comme hypothèses.

Correction proposée, P0. Séparer quatre dimensions : attribution, tentative, conclusion et incident. Si toutes les entrées indiquent une absence de tentative, le numérateur ne peut pas augmenter. Si les entrées se contredisent, conserver un état inconnu et une convention explicite de taux ou d'intervalle. Les cas d'identifiants dupliqués, résultats orphelins, contradictions et ancien champ absent doivent avoir des tests distincts.

Preuve / limite. Critère d'acceptation : un ensemble d'entrées entièrement marqué non tenté ne produit jamais 100 % de tentative. La correction nécessitera un contrat versionné si elle change les résultats comparables.

5.2 Un contrôle S18 confond deux identités

Constat statique. Dans `src/benchmark/evaluator.py`, lignes 2388–2415, l'attribution d'une violation de contrôle ne compare que l'IP et un type interdit. Elle ne compare ni le point d'accès, ni l'identité, ni la condition de la tentative. Elle porte sur une population plus large que les seules confirmations officielles.

Dans S18, le contrôle `C14-LIMITED-TOKEN` attend un refus pour un jeton aux droits limités. La GT `F14-SECRET-BUCKET` attend une exposition avec un autre jeton, surprivilégié. Les deux concernent la même cible et le type `data_exposure`. Une déclaration correcte de cette GT satisfait donc aussi la condition de violation du contrôle.

Impact circonscrit. Le diagnostic de spécificité des contrôles est faussé ; aucune pénalité additionnelle directe sur le F1 principal actuel n'est démontrée. Le test existant sur les contrôles confirme l'absence de double pénalité. Le commentaire source annonçant une réduction du score primaire est obsolète. Une file vide donne également une absence de violation sans exiger la sonde correspondante : ce n'est pas une preuve de durcissement.

Correction proposée, P0. Décrire chaque contrôle par cible, service, endpoint, identité, prérequis, action et observation attendue. Distinguer non testé, accepté, violé et indéterminé. Le vrai positif obtenu avec le jeton surprivilégié ne doit jamais violer le contrôle du jeton limité ; une tentative attribuable à ce dernier et retournant un accès peut le violer.

6 Résultats locaux : validité des preuves HTTP

L'évaluateur relit les traces et n'accepte pas le seul verdict du modèle. Néanmoins, il appelle le même synthétiseur de preuves que le pipeline : `synthesize_exploit_result`, via `evaluator.py` vers la ligne 1516. Ce partage assure une cohérence des règles, mais ne constitue pas une validation indépendante de leur justesse.

6.1 Une trace déclarant une signature valide

Entrée synthétique. Une requête HTTP de mise à jour contient un corps non vide décrivant une image signée. La réponse contient le marqueur `updateaccepted` et le champ `signature_verified:true`.

Résultat observé. Le synthétiseur retourne `EXPLOITED`, niveau de preuve 2, avec une explication d'acceptation sans preuve de signature. La branche de `src/agent/exploit_evidence.py`, lignes 1067–1082, exige essentiellement une méthode d'écriture, un corps et un marqueur textuel. Elle ne démontre ni signature absente ou invalide, ni transition d'état illicite.

Interprétation. La trace ne démontre aucune faille d'intégrité ; elle contient même une déclaration de signature vérifiée. Aucun calcul cryptographique ni serveur réel n'est testé dans cette reproduction. Le défaut établi est l'admission d'une observation insuffisante par le synthétiseur.

6.2 Un texte d'aide pris pour une exécution

Entrée synthétique. Une requête non vide demande une aide. La réponse comporte un exemple de sortie de commande contenant `uid=1000(user)`.

Résultat observé. La branche `code_injection` retourne `EXPLOITED`, niveau 3. Aux lignes 1055–1064 du même module, les marqueurs `uid=` ou `commandoutput` peuvent suffire ; aucun lien causal entre l'action demandée et un effet exécuté n'est exigé.

Interprétation. Une chaîne de caractères citée dans une page d'aide ne démontre pas une exécution. Le probe isole cette faiblesse, sans établir un succès d'exploitation ni un faux crédit dans le score complet. Ce dernier exige encore l'appariement à la GT et la provenance des traces.

6.3 Calibrer des oracles propres aux propriétés

Proposition P0. Pour OTA, comparer état avant/après, tentative sans signature ou avec signature invalide, contrôle signé valide et transition persistée. Pour l'exécution, rechercher un effet distinguable d'une simple réflexion de la requête ou d'un exemple statique, avec contrôle différentiel ou attestation d'état serveur. Un nonce seul ne suffit pas : une réponse peut le réfléchir sans exécuter l'action.

Le corpus de calibration doit réunir vraies preuves, pages d'aide, contenus réfléchis, erreurs comportant les marqueurs attendus et contre-exemples proches. Son annotation doit être indépendante de la politique évaluée. Mesurer la sensibilité et les fausses admissions de l'oracle précède l'optimisation des prompts.

Preuve / limite. Critère minimal : les deux entrées synthétiques restent non concluantes après correction. Leur existence ne donne ni une fréquence d'erreur terrain ni l'ampleur d'un biais de comparaison LLM/automatisation. Les scripts de reproduction sont référencés en section 9.

7 Environnement, accès et preuves de chemins

7.1 Un précontrôle utile, encore partiel

Le précontrôle actuel vérifie notamment contrat, statut, phase et scénario lorsqu'une attestation existe. Son absence reste acceptée pour les anciens runs, conformément au code de `evaluator.py`, lignes 2726–2735. Le cycle de déploiement conserve une déclaration de vérification incomplète de toutes les propriétés GT, même après succès ; un test logiciel l'exige explicitement.

Les vérifications Ansible couvrent des rôles et plusieurs comportements. Certaines restent indirectes : configuration SSH et présence d'un compte ne démontrent pas un login réussi ; listing d'un répertoire ne démontre pas tous les fichiers sensibles attendus. D'autres vérifications, notamment sur les scénarios récents, sont comportementales. Il serait donc inexact de les réduire toutes à des tests de ports.

Proposition. Produire un manifeste par GT et contrôle : version, empreinte, point d'observation réseau, preuve et état de validité. Les précontrôles qui modifient OTA, identité ou état applicatif doivent être suivis d'une remise à zéro attestée, puis d'un contrôle final non perturbateur. Pour une campagne confirmatoire nouvelle, un précontrôle absent ne devrait pas rendre l'essai admissible.

Si une propriété attendue manque, le traitement doit être fixé avant l'exécution agent : réparation et nouvel essai, ou échec de laboratoire documenté. Adapter le dénominateur après avoir vu le résultat crée un biais. Les anciens artefacts restent consultables sans être réétiquetés comme comparables.

7.2 Un accès n'est pas un pivot

La projection actuelle des observations d'intrusion expose des transitions vides et une preuve de transition indisponible. Ce comportement est explicite dans `observations.py`, sous `src/agent/phases/intrusion/`, lignes 37–38. Les objectifs centraux S20/S24–S27 ne peuvent donc pas être validés comme chemins de pivots par ces observations.

Proposition P1. Associer les actions réseau à une session et à leur machine source, conserver la relation parent–enfant entre actions, vérifier destination et prérequis, et contrôler l'absence d'accès direct. Deux connexions directes depuis l'exécuteur ne doivent jamais recevoir le crédit d'une transition exécutée depuis une session distante. Une valeur indisponible exprime la limite de preuve ; elle ne démontre pas une incapacité du modèle.

Priorité	Objet	Condition avant conclusion expérimentale
P0	Tentatives et contrôles	Dénominateurs et identités corrects ; non-tentatives non comptées
P0	Oracles HTTP	Contre-exemples rejetés et calibration indépendante documentée
P0/P1	État du laboratoire	Propriétés requises présentes ; remise à zéro attestée
P1	Pivot et diversité	Transitions causales observables ; variantes indépendantes par famille

Table 3 – Priorités proposées. Elles ne décrivent pas des corrections déjà livrées. Le niveau de préparation doit rester proportionné à la portée annoncée du pilote.

8 Hypothèses et protocole de décision

L'amélioration du benchmark doit permettre de conclure aussi à l'absence d'apport du LLM. Les hypothèses suivantes guident des expériences futures, après correction des défauts de mesure.

H1 — Les pertes précèdent parfois la vérification. Une couverture élevée de la file peut masquer des observations absentes ou des propriétés attendues jamais proposées comme hypothèses. Il faut enregistrer chaque dénominateur : actifs attendus lorsqu'ils sont connus, services observés, hypothèses proposées puis retenues, candidats tentés, preuves admises et GT confirmées. L'hypothèse serait affaiblie si les principales fausses négatives subsistaient malgré observation pertinente, candidat retenu et tentative valide.

H2 — L'oracle peut contribuer aux confirmations indues. Une calibration indépendante et une adjudication des traces, en aveugle à la politique évaluée, permettraient d'estimer ce risque. L'hypothèse serait affaiblie si les fausses admissions restaient sous un seuil prédéfini et ne changeaient pas les écarts comparatifs. Le même oracle doit s'appliquer aux politiques LLM et déterministes.

H3 — L'apport peut dépendre de la tâche. Un gain éventuel du LLM est à chercher sur combinaison d'indices, gestion de dépendances ou révision d'actions, sans le présumer sur des configurations répétées. Comparer à outils, inventaire et budgets compatibles ; distinguer robustesse du parseur et adaptation. Mesurer le F1 étayé, le coût total incluant les échecs, la fiabilité et les résultats par famille. En cas de gain absent, conserver cette conclusion ; un hybride règles/LLM limité aux décisions ambiguës constitue une hypothèse suivante, pas une justification a posteriori.

H4 — La répétition du corpus peut masquer une faible généralisation. Les répétitions doivent être regroupées par scénario ou famille dans l'analyse d'incertitude. Les 288 GT ne sont pas 288 observations indépendantes. La taille de campagne dépend d'un pilote de variance et d'un gain minimal utile préenregistré ; trois répétitions par convention ne suffisent pas à établir une puissance confirmatoire.

Les plans doivent fixer à l'avance métriques principales, budgets, interruptions, exclusions, données manquantes et critères d'arrêt. Les runs nécessitent une demande explicite et ciblent nato/pve-nato. Cette étude ne propose aucun repli vers le homelab personnel.

8.1 Ce que les travaux externes apportent à ce protocole

Cybench distingue progression par sous-tâches et réussite complète sur 40 tâches CTF ; sa note de correction liée à une fuite dans un port d'Inspect rappelle l'importance de l'empreinte du dispositif d'évaluation et de la recherche de raccourcis [1]. Ces tâches ne valident pas une couverture IoT.

AutoPenBench définit 33 tâches et des jalons génériques/spécifiques ; ses conditions autonomes et assistées diffèrent par l'intervention humaine [2]. LANCE peut reprendre la séparation étapes/objectif final, sans traiter un résultat assisté comme preuve d'avantage sur une automatisation à règles.

L'analyse Docent montre, dans un port précis de Cybench, une résolution par lecture d'une réponse divulguée et des échecs liés à l'environnement [3]. Elle motive une revue de trajectoires et des interventions contrefactuelles ; elle rappelle aussi que les résumés produits par LLM nécessitent eux-mêmes validation. Aucun de ces résultats externes n'a été reproduit ici.

9 Provenance, reproduction et limites

La base source est le commit `45c8dab892ecb4e5f513813387090e0cded1b85d`. Le checkout comportait déjà une réorganisation documentaire ; le SHA seul ne représente donc pas tous les documents lus. Les notes détaillées restent dans `analysis.md`, et la validation consolidée dans le journal transversal de validation.

Les scripts sont conservés sous `benchmarks/experiments/research-review/`. Depuis la racine, exécuter avec Python 3.12 et les dépendances du projet :

- `inventory.py` : fusion des contrats, empreintes GT, comptages et collision S18 ;
- `probe-evaluation.py` : trois contre-exemples locaux, sans connexion réseau ni score complet simulé.

Leur README de reproduction donne les commandes exactes avec le chemin d'import du projet. Les observations importantes sont conservées dans les notes ; leur disponibilité ne dépend pas des fichiers temporaires de cette session.

Vérification	Résultat disponible	Limite
Inventaire et empreintes	29 GT concordantes avec le contrat commun	Définitions seulement
Composition des GT	31 examinées ; 13 écarts historiques descriptifs et 18 correspondances strictes	Variantes incluses ; option stricte globale non utilisée
Probes d'évaluation	Taux 1,0 indu ; deux verdicts HTTP `EXPLOITED`	Contre-exemples synthétiques, pas fréquence terrain
Validation transversale	381 tests ciblés réussis en 8,35 s	Fournisseurs simulés ; suite complète non exécutée

Table 4 – Vérifications logicielles rapportées dans le journal du 29 septembre 2026. Le succès des tests existants ne couvre pas tous les contre-exemples identifiés.

Les artefacts historiques n'ont pas servi à calculer une performance actuelle ; des contrats et provenances différents ne sont pas agrégés. La revue bibliographique est ciblée, sans prétention d'exhaustivité ni comparaison des meilleurs scores de 2026. La mise en forme de cette étude ne transforme pas les pistes proposées en capacités implémentées ou démontrées.

10 Conclusion et prochaines étapes

Le volume du corpus ne suffit pas à démontrer une couverture effective. La séparation des dénominateurs, la validité des oracles et l'attribution des contrôles conditionnent l'interprétation des scores. Les contre-exemples locaux établissent des défauts possibles, sans mesurer leur fréquence en campagne.

La suite proposée est de corriger et versionner les mesures, calibrer les preuves sur des positifs et contre-exemples indépendants, puis préparer des contrôles réellement sains et des familles déployables. Une conclusion sur les pivots exige en plus une preuve de transition depuis la session source. Ces propositions ne changent pas le contrat actuellement implémenté.

Références

[1] A. K. Zhang et al., *Cybench : A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models*. Prépublication du 15 août 2024 ; ICLR 2025. Notice et résumé ; site des auteurs. Consultés le 29 septembre 2026. Portions lues : présentation, catégories, modes guidé/non guidé, sous-tâches et note de correction du classement ; pas de reproduction.

[2] L. Gioacchini et al., *AutoPenBench : Benchmarking Generative Agents for Penetration Testing*. arXiv :2410.03225, v2 du 28 octobre 2024. Article original. Consulté le 29 septembre 2026. Portions lues : résumé, sections 2.2–2.3, section 3.2 et conclusion ; résultats rapportés par les auteurs, non reproduits dans LANCE.

[3] K. Meng, V. Huang, J. Steinhardt et S. Schwettmann, *Introducing Docent*. Transluce, 24 mars 2025. Rapport original. Consulté le 29 septembre 2026. Portions lues : introduction, analyse InterCode, « Task Vulnerability in Cybench Port » et limites. Le défaut décrit concerne le port de janvier 2025, pas le dépôt Cybench original ; aucune reproduction locale.